

Prelims

For this Ben Olschar, 108021211678 and Frederik Hüttemann, 108021215247 cooperated with each other. We did implement the code on our own, but agreed to using one version. The discussion is formulated together.

Discussion

Here, we will discuss the results from the different tests.

Task 2

In task 2 we had to compare the execution time between the shared and non-shared algorithm. For the CPU, static memory was used. Here, the shared algorithm takes up more time than the non-shared one as the shared algorithm has multiple memory accesses which are slow on CPU.

On GPU however, we can use the fast shared memory as well as the possibility to run the algorithm in parallel. The shared algorithm on GPU is about 1.55 times faster than the non-shared algorithm. To mitigate noise, the test has been run for 100 times. This speedup is because of reusing results by storing and accessing them in fast shared memory. This leads to less accesses to memory outside the shared memory.

Task 3

As expected the CPU algorithm is super slow for large matrices while the GPU version handles large dimensions pretty well. While the CPU version takes about 4 hours to compute, the GPU version is finished after 1,2 seconds. This is a speed-up of a factor of about 12.000. This is because of parallelism. While the CPU has to do all operations in sequence, the GPU can handle many calculations at the same time.

Task 4

This task contained two subparts. The first part should analyze the dependence of TILEWIDTH to the execution time. The second should relate the matrix size to the speed-up factor between CPU and GPU.

Task 4.1

To show the dependence on TILEWIDTH, the code has been recompiled with different values for the TILEWIDTH. We can see, that with an increasing TILEWIDTH, the execution time is reduced. The best performance is achieved for a TILEWIDTH of 32, which is also the limit, as we do start $32 \times 32 = 1024$ threads, which is the limit on a block. What can also be observed is, that we already achieve a similar execution time for a TILEWIDTH of 26. The reason for the best performance when using a TILEWIDTH of 32 is because again the number of slow memory

accesses is reduced to the minimum because the shared memory storage is as large as possible.

Task 4.2

To show the dependence of speedup factor of GPU vs CPU on matrix size, different matrix sizes have been tested. As a metric for size, the resulting dimension has been taken. We can see, that the speedup factor increases monotonically with the matrix size. This is because the overhead for GPU calculations are mitigated for larger matrices while the CPU's performance remains limited due to its sequential operations. The GPU therefore can utilize the parallelism much better for larger problem sizes.

Task 5

There are no conflicts during loading the data to the shared memory. This is because, every float takes up space of one bank as they are 4-byte floats. Therefore, every float has its own bank. Each warp consists of at most TILEWIDTH threads, which do have a unique threadIdx in x, as they are linearly aligned.

On accessing the memory again, we do have to look at the indexing again. For a warp, threads have a different tx, but the same ty, as they the linear indices for threads of a warp are consecutive. So for the access on `Nds[k][tx]`, we know that tx are different across a warp and therefore the read of elements is consecutive across the bank. For accessing `Mds[ty][k]`, we know that ty is the same for all threads of a warp. Because we use the inner loop for indexing k, we know, that we access the same k for all threads, meaning we are trying to read the same bank data from multiple threads. According to the lecture, this is not considered as a bank hit, as the hardware does handle multiple accesses to the same bank differently.